

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1153

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (BL5)

Assessment Tool Weightage: 30%

QUESTIONS: 1

Total Marks:10

Annexure A

Reverse Engineering Task

Question 2: GoF Design Pattern Refactoring

GoF Patterns | Flexibility | Maintainability | Code Reuse

Bloom's Level: BL4 – Analyse & Apply

Scenario:

A legacy application does not use any design patterns, leading to repetitive code and difficulty in extending features. New requirements force developers to modify existing classes every time, violating the Open/Closed Principle. Code duplication exists across modules, and adding new behaviour requires significant rework.

Question (Marks: 30):

Analyze and refactor the system by introducing appropriate GoF design patterns, and justify how they improve flexibility and maintainability.

Code of Conduct:

I Y. Abhay-(192411219) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Y. Abhay

RUBRICS FOR CALCULATION

Reverse Engineering Task

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Question 2: GoF Design Pattern Refactoring

GoF Patterns | Flexibility | Maintainability | Code Reuse

Bloom's Level: BL4 – Analyse & Apply

Introduction

The legacy application has several design problems:

- Repetitive code across modules
- High coupling between classes
- Violation of Open/Closed Principle (OCP)
- Difficult feature extension
- Poor maintainability
- Low code reuse

To solve these issues, the application can be refactored using **Gang of Four (GoF) Design Patterns**. These patterns provide reusable solutions that improve software flexibility, maintainability, and scalability.

1. Analysis of Existing System

Current Problems

New Requirement



Modify Existing Classes



Retest Entire System



More Bugs and Complexity

Effects

- Adding new functionality requires changing existing code.
- Duplicate code appears in multiple modules.
- Maintenance becomes expensive.
- Testing effort increases.
- Scalability is limited.

2. Refactoring Using GoF Design Patterns

A. Strategy Pattern

Problem

Different algorithms (payment methods, discount calculations, shipping methods) are implemented using multiple if-else statements.

Before Refactoring

```
PaymentProcessor
├── if(Card)
├── if(UPI)
└── if(NetBanking)
```

Adding a new payment method requires modifying the existing class.

After Refactoring

```
PaymentStrategy
  ^
-----
|   |   |
Card UPI NetBanking
```

Benefits

- New payment methods can be added easily.
- Supports Open/Closed Principle.
- Reduces conditional complexity.
- Improves flexibility.

B. Factory Method Pattern

Problem

Object creation logic is repeated throughout the system.

Before Refactoring

```
if(type=="Admin")
    create Admin

if(type=="Customer")
    create Customer
```

After Refactoring

```
UserFactory  
|  
+---- createUser()
```

Benefits

- Centralized object creation.
- Easy addition of new object types.
- Better maintainability.

C. Singleton Pattern

Problem

Multiple database connection objects waste resources.

Solution

```
DatabaseConnection  
|  
+---- getInstance()
```

Only one instance of the database connection exists.

Benefits

- Efficient memory usage.
- Improved performance.
- Consistent database access.
- Reduced resource consumption.

D. Observer Pattern

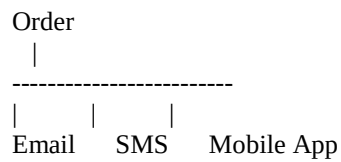
Problem

Notification modules are tightly coupled with business logic.

Example

Order status changes should notify multiple users.

Refactored Design



Whenever an order changes, all registered observers are notified automatically.

Benefits

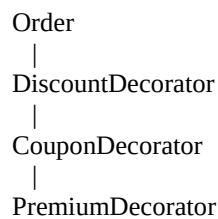
- Loose coupling.
- Easy addition of new notification channels.
- Better extensibility.

E. Decorator Pattern

Problem

Adding discounts, coupons, and premium features creates many subclasses.

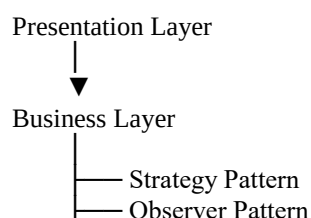
Refactored Design

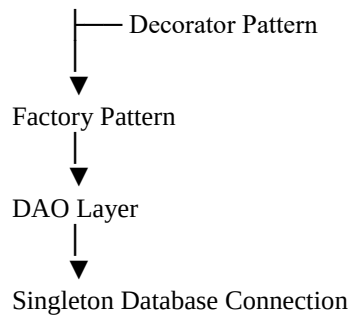


Benefits

- Dynamic feature addition.
- No modification of existing classes.
- Better code reuse.

3. Refactored Architecture





4. Open/Closed Principle Compliance

Before

Add New Feature
↓
Modify Existing Code
↓
Higher Risk of Errors

After

Add New Class
↓
Integrate with Existing System
↓
No Changes to Existing Code

The application becomes **Open for Extension** and **Closed for Modification**.

5. Improvements Achieved

Aspect	Before Refactoring	After Refactoring
Flexibility	Low	High
Maintainability	Difficult	Easy
Code Reuse	Low	High
Scalability	Limited	Excellent
Coupling	High	Low
Extensibility	Difficult	Easy
Testing	Complex	Simpler

6. How Flexibility is Improved

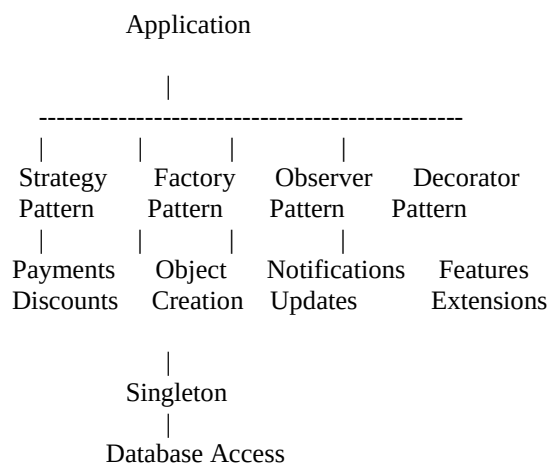
- New payment methods can be added using Strategy Pattern.
- New object types can be created using Factory Method.
- New notification services can be added using Observer Pattern.
- New features can be attached dynamically using Decorator Pattern.

No existing code requires modification.

7. How Maintainability is Improved

- Clear separation of responsibilities.
- Reduced code duplication.
- Smaller and focused classes.
- Easier debugging and testing.
- Faster implementation of future enhancements.

Diagram of Refactored System



Conclusion

The legacy application can be successfully refactored using **GoF Design Patterns** such as **Strategy, Factory Method, Singleton, Observer, and Decorator**. These patterns eliminate code duplication, reduce coupling, and enforce the **Open/Closed Principle**. As a result, the system becomes more flexible, maintainable, reusable, and scalable, enabling developers to add new features with minimal effort and without modifying existing code.